Template Editor Dialog - テンプレート編集ダイアログ

概要

目的

MAIML Petri Netエディタ内で、Material/Condition/ResultテンプレートのProperties、Contents、Insertion、PlaceRefsなどを編集するためのモーダルダイアログです。NestedPropertyEditorを使用してネストされたプロパティ構造を階層的に編集できます。

使用場面

- Petri Netエディタ画面でTemplateを持つPlaceノードをダブルクリックした時
- 新規テンプレートを作成する時
- 既存テンプレートを編集する時
- テンプレートファイルから名前空間を取り込む時

アクセス権限

- **実験管理者（Experiment Admin）**: 利用可能
- MAIML編集権限を持つユーザーのみアクセス可能

画面構成

レイアウト構造

```
┌──────────────────────────────────────────────────────┐
│ [アイコン] {nodeCategory}を編集          [×]          │
├──────────────────────────────────────────────────────┤
│                                                      │
│ [⚠️ バリデーション警告（N件）]  ※エラーがある場合      │
│                                                      │
```


📋 基本情報
・テンプレートID
・UUID [生成ボタン]
・Name
・Description
🔗 Place References [追加]
・Place Ref一覧（削除ボタン付き）
Insertion情報 ※Material/Resultのみ
・URI
・Hash Method
・Hash Value
・Format
Properties
[新規プロパティ追加] [テンプレートから取り込み]
NestedPropertyEditor（プロパティ階層表示）
- key1: value1
- childKey1: childValue1
- childKey2: childValue2
- key2: value2
[🔗 継承元プロパティ（親テンプレート）] ※継承時のみ
（Collapse展開で親テンプレートのPropertiesを表示）
Contents ※Resultテンプレートのみ
[新規Content追加] [テンプレートから取り込み]
NestedPropertyEditor（Content階層表示）
[キャンセル] [💾 保存]

UI要素一覧

1. ヘッダー

- タイトル: `{アイコン} {nodeCategory}を編集`
  - Material: 🧪 (ExperimentOutlined)
  - Condition: ⚙️ (SettingOutlined)
  - Result: 📊 (BarChartOutlined)
- 閉じるボタン（×）

2. バリデーション警告（エラーがある場合のみ表示）

- Alert（warning）
- エラー件数とメッセージのリスト表示

3. 基本情報セクション（薄青背景）

- **テンプレートID**: Input（必須）
- **UUID**: Input + [生成]ボタン（ThunderboltOutlined）
- **Name**: Input
- **Description**: TextArea（2行）

4. Place Referencesセクション

- 見出し: "🔗 Place References"
- [追加]ボタン（PlusOutlined）
- Place Ref一覧カード
  - Place Ref番号
  - Ref ID表示
  - [削除]ボタン（DeleteOutlined）

5. Insertionセクション（Material/Resultのみ）

- 見出し: "👣 Insertion情報"
- **URI**: Input

- **Hash Method**: Input
- **Hash Value**: Input
- **Format**: Input

6. Propertiesセクション

- 見出し: "🏷️ Properties"
- [新規プロパティ追加]ボタン（PlusOutlined）
- [テンプレートから取り込み]ボタン（DownloadOutlined）
- NestedPropertyEditor コンポーネント（プロパティごと）
  - 階層的なプロパティ表示
  - 展開/折りたたみ機能
  - 属性・要素の編集
  - 子要素の追加・削除

7. 継承元プロパティセクション（継承時のみ）

- Collapse（折りたたみ可能）
- 見出し: "🔗 継承元プロパティ（親テンプレート: {id}）"
- Tag: "読み取り専用"（青）
- Alert: "このプロパティは親テンプレートから継承されています"
- 親テンプレートのProperties一覧（読み取り専用）

8. Contentsセクション（Resultテンプレートのみ）

- 見出し: "🏷️ Contents"
- [新規Content追加]ボタン（PlusOutlined）
- [テンプレートから取り込み]ボタン（DownloadOutlined）
- NestedPropertyEditor コンポーネント（Contentごと）

9. フッター

- [キャンセル]ボタン
- [💾 保存]ボタン（primary）

機能仕様

Props定義

```
interface TemplateEditorDialogProps {
  open: boolean;                          // ダイアログの開閉状態
  node: {                                 // 編集対象ノード
    id: string;
    data: NodeData;
  };
  onClose: () => void;                    // 閉じる時のコールバック
  onSave: (                               // 保存時のコールバック
    nodeId: string,
    templateRef: Template,
    isNewTemplate: boolean,
    templateCategory: string,
    isCommonTemplate: boolean,
    methodId: string | null
  ) => void;
  allNodes?: Array<{ id: string; data: NodeData }>; // 全ノード（PlaceRef追加用）
  templateErrors?: Record<string, TemplateError[]>; // バリデーションエラー
}
```

State管理

基本情報State

```
const [hasChanges, setHasChanges] = useState(false);
const [templateId, setTemplateId] = useState('');
const [uuid, setUuid] = useState('');
const [name, setName] = useState('');
const [description, setDescription] = useState('');
const [placeRefs, setPlaceRefs] = useState<PlaceRef[]>([]);
const [insertion, setInsertion] = useState<Insertion>({
  uri: '',
  hash: { method: '', value: '' },
  format: ''
});
```

Properties編集State

```
const [properties, setProperties] = useState<Record<string, PropertyData>>({});
const [expandedNodes, setExpandedNodes] = useState(new Set<string>());
const [showAddModal, setShowAddModal] = useState(false);
const [newPropertyKey, setNewPropertyKey] = useState('');
const [newPropertyTagName, setNewPropertyTagName] = useState('property');
const [forceRenderKey, setForceRenderKey] = useState(0);
```

Contents編集State

```
const [contents, setContents] = useState<Record<string, PropertyData>>({});
const [expandedContentNodes, setExpandedContentNodes] = useState(new Set<string>());
const [showContentAddModal, setShowContentAddModal] = useState(false);
const [newContentKey, setNewContentKey] = useState('');
const [newContentTagName, setNewContentTagName] = useState('content');
const [forceContentRenderKey, setForceContentRenderKey] = useState(0);
```

PlaceRef編集State

```
const [showPlaceRefAddDialog, setShowPlaceRefAddDialog] = useState(false);
```

名前空間取り込みState

```
const [showNamespaceDialog, setShowNamespaceDialog] = useState(false);
const [formatsResponse, setFormatsResponse] = useState<ListFormatsResponse>({ departments: [] });
const [selectedNamespaceFile, setSelectedNamespaceFile] = useState<string>('');
const [isLoadingTemplates, setIsLoadingTemplates] = useState(false);
const [importedNamespaces, setImportedNamespaces] = useState<string[]>([]);
const [namespaceImportMode, setNamespaceImportMode] = useState<'replace_all_properties' |
'replace_all_contents'>('replace_all_properties');
const [namespacePreview, setNamespacePreview] = useState<{
  parsedData?: NamespaceData;
  xmlPreview?: string;
  error?: string;
}>({});
```

```
const [showConfirmDialog, setShowConfirmDialog] = useState(false);
const [duplicateKeys, setDuplicateKeys] = useState<string[]>([]);
```

バリデーションState

```
const [validationErrors, setValidationErrors] = useState<string[]>([]);
```

useMemo - 計算値

nodeCategory（テンプレートタイプ判定）

```
const nodeCategory = useMemo(() => {
  if (!node) return '';
  return node.data.placeType; // 'material' | 'condition' | 'result'
}, [node]);
```

templateIcon（テンプレートアイコン）

```
const templateIcon = useMemo(() => {
  if (!node) return <ExperimentOutlined />;
  return node.data.placeType === 'material' ? <ExperimentOutlined /> :
         node.data.placeType === 'condition' ? <SettingOutlined /> :
         <BarChartOutlined />;
}, [node]);
```

mergedTemplate（マージされたテンプレートデータ）

```
const mergedTemplate: Template | null = useMemo((): Template | null => {
  if (!node) return null;

  const template = (node.data.template || {}) as Partial<Template>;
  const templateRef = (node.data.templateRef || {}) as Partial<Template>;

  // template と templateRef をマージして返す
  return {
```

```
    id: template.id || templateRef.id || '',
    uuid: template.uuid || templateRef.uuid || '',
    name: template.name || templateRef.name || '',
    description: template.description || templateRef.description,
    category: template.category || templateRef.category || '',
    properties: template.properties || templateRef.properties || [],
    contents: template.contents || templateRef.contents || [],
    insertion: template.insertion || templateRef.insertion,
    placeRefs: template.placeRefs || templateRef.placeRefs || [],
    templateRef: template.templateRef,
  };
}, [node]);
```

inheritedTemplate（継承元テンプレート）

```
const inheritedTemplate = useMemo((): Template | null => {
  if (!mergedTemplate?.templateRef) return null;

  const parentTemplateId = mergedTemplate.templateRef.ref;
  if (!parentTemplateId || typeof parentTemplateId !== 'string') return null;

  // window.clineAllTemplatesから検索
  if (window.clineAllTemplates) {
    const category = node.data.placeType === 'material' ? 'material' :
                     node.data.placeType === 'condition' ? 'condition' : 'result';
    const templates = window.clineAllTemplates[category];

    if (templates.length > 0) {
      const found = templates.find((t: Template) => t.id === parentTemplateId);
      if (found) return found as Template;
    }
  }

  // allNodesから検索（フォールバック）
  const parentNode = allNodes.find(n =>
    n.id === parentTemplateId ||
    n.data?.template?.id === parentTemplateId ||
    n.data?.templateRef?.id === parentTemplateId
  );

  if (parentNode) {
    return (parentNode.data.template || parentNode.data.templateRef) as Template;
  }

  return null;
}, [mergedTemplate, allNodes, node]);
```

カスタムフック

useNamespaceOperations

```
const { analyzeNamespaceFile, replaceTemplateWithNamespace } = useNamespaceOperations();
```

- `analyzeNamespaceFile(fileKey: string)`: テンプレートファイルを解析して名前空間データを取得
- `replaceTemplateWithNamespace(parsed, type, setEditingTemplate)`: 名前空間データでテンプレートを置き換え

主要機能

1. 初期化処理（useEffect）

ダイアログが開かれた時、ノードデータからStateを初期化します。

```
useEffect(() => {
  if (open && node && mergedTemplate) {
    // 基本情報
    setTemplateId(mergedTemplate.id || '');
    setUuid(mergedTemplate.uuid || '');
    setName(mergedTemplate.name || '');
    setDescription(mergedTemplate.description || '');
    setInsertion(mergedTemplate.insertion || { uri: '', hash: {method: '', value: ''}, format: ''
});

    // PlaceRefs
    const loadedPlaceRefs = mergedTemplate.placeRefs || [];
    setPlaceRefs(loadedPlaceRefs);

    // Properties変換（配列 → NestedPropertyEditor形式）
    let convertedProperties: Record<string, PropertyData> = {};
    if (mergedTemplate.properties) {
      if (Array.isArray(mergedTemplate.properties)) {
        convertedProperties = convertPropertiesArrayToNestedFormat(mergedTemplate.properties);
      } else if (typeof mergedTemplate.properties === 'object') {
        // 既にNestedPropertyEditor形式かチェック
        const firstKey = Object.keys(mergedTemplate.properties)[0];
```

```
        const firstProp = mergedTemplate.properties[firstKey] as Record<string, unknown>;
        const isAlreadyNestedFormat = firstProp &&
          typeof firstProp === 'object' &&
          ('attributes' in firstProp || 'elements' in firstProp || 'children' in firstProp);

        if (isAlreadyNestedFormat) {
          convertedProperties = mergedTemplate.properties as Record<string, PropertyData>;
        }
      }
    }
    setProperties(convertedProperties);

    // Contents変換（配列 → NestedPropertyEditor形式）
    let convertedContents: Record<string, PropertyData> = {};
    if (mergedTemplate.contents) {
      if (Array.isArray(mergedTemplate.contents) && mergedTemplate.contents.length > 0) {
        convertedContents = convertContentsArrayToNestedFormat(mergedTemplate.contents);
      }
    }
    setContents(convertedContents);

    // 展開状態とレンダリングキーをリセット
    setExpandedNodes(new Set());
    setExpandedContentNodes(new Set());
    setForceRenderKey(Date.now());
    setForceContentRenderKey(Date.now());
  }
}, [open, node, mergedTemplate]);
```

2. UUID生成

```
const handleGenerateUuid = () => {
  const newUuid = crypto.randomUUID();
  setUuid(newUuid);
  setHasChanges(true);
};
```

ボタンクリックで新しいUUID（urn:uuid:形式）を自動生成します。

3. Properties編集機能

3-1. 新規プロパティ追加

```
const handleAddProperty = () => {
  if (!newPropertyKey.trim()) {
    message.error('プロパティキーを入力してください');
    return;
  }

  if (properties[newPropertyKey.trim()]) {
    message.error('同じキーのプロパティが既に存在します');
    return;
  }

  setProperties({
    ...properties,
    [newPropertyKey.trim()]: {
      key: newPropertyKey.trim(),
      tagName: newPropertyTagName,
      attributes: {},
      elements: {},
      children: []
    }
  });

  setNewPropertyKey('');
  setNewPropertyTagName('property');
  setShowAddModal(false);
};
```

3-2. ネスト子要素追加

```
const handleAddNestedChild = (parentKey: string, childTagName: string, specifiedKey?: string) =>
{
  // ユーザー指定のkeyを優先、なければランダム生成
  const childKey = specifiedKey && specifiedKey.trim()
    ? specifiedKey.trim()
    : `${childTagName}_${Date.now()}`;

  // ✅ 重複keyチェック
  const hasDuplicate = checkDuplicateKey(properties, parentKey, childKey);
  if (hasDuplicate) {
    message.error(`同じ階層に既に「${childKey}」が存在します`);
    return;
  }

  // 再帰的に親を検索して子要素を追加
  setProperties(prevProperties => {
    const updated = JSON.parse(JSON.stringify(prevProperties));
```

```
    const found = findAndUpdateParent(updated, parentKey);
    return updated;
  });

  setForceRenderKey(Date.now()); // 強制再レンダリング
  setExpandedNodes(prev => new Set([...prev, parentKey])); // 自動展開
  setHasChanges(true);
};
```

3-3. 属性更新

```
const handleUpdateNestedAttribute = (propertyKey: string, attrName: string, attrValue: string) =>
{
  setProperties(prevProperties => {
    const updated = JSON.parse(JSON.stringify(prevProperties));
    findAndUpdate(updated, propertyKey); // 再帰的に検索して更新
    return updated;
  });

  setForceRenderKey(Date.now());
  setHasChanges(true);
};
```

3-4. 要素更新

```
const handleUpdateNestedElement = (propertyKey: string, elemName: string, elemValue: string) => {
  // handleUpdateNestedAttributeと同様のパターン
};
```

3-5. プロパティ削除

```
const handleDeleteProperty = (propertyKey: string) => {
  const updated = { ...properties };
  delete updated[propertyKey];
  setProperties(updated);
  setHasChanges(true);
};
```

3-6. 展開/折りたたみ

```
const handleToggleExpanded = (nodeId: string) => {
  setExpandedNodes(prev => {
    const next = new Set(prev);
    if (next.has(nodeId)) {
      next.delete(nodeId);
    } else {
      next.add(nodeId);
    }
    return next;
  });
};
```

4. Contents編集機能（Resultテンプレートのみ）

Propertiesと同様のハンドラー群を持ちます：

- `handleAddContent()`
- `handleAddNestedContentChild()`
- `handleUpdateContentAttribute()`
- `handleUpdateContentElement()`
- `handleDeleteContent()`
- `handleToggleContentExpanded()`

5. PlaceRef編集機能

5-1. PlaceRef追加ダイアログを開く

```
setShowPlaceRefAddDialog(true);
```

5-2. PlaceRef追加

```
// PlaceRef追加ダイアログ内のボタンクリック
onClick={() => {
  setPlaceRefs([...placeRefs, {id: '', ref: place.id}]);
  setShowPlaceRefAddDialog(false);
  setHasChanges(true);
  message.success('Place Referenceを追加しました');
}}
```

5-3. PlaceRef削除

```
onClick={() => {
  const updated = placeRefs.filter((_, i) => i !== index);
  setPlaceRefs(updated);
  setHasChanges(true);
}}
```

6. 名前空間取り込み機能

6-1. テンプレートファイル一覧取得

```
const handleOpenNamespaceImport = async (mode: 'replace_all_properties' | 'replace_all_contents'
= 'replace_all_properties') => {
  setNamespaceImportMode(mode);
  setIsLoadingTemplates(true);
  setShowNamespaceDialog(true);

  try {
    const result = await fetchTemplateFiles();
    setFormatsResponse(result);
  } catch (error) {
    message.error('テンプレートファイル一覧の取得に失敗しました');
    setFormatsResponse({ departments: [] });
  } finally {
    setIsLoadingTemplates(false);
  }
};
```

6-2. ファイル選択時のプレビュー取得

```
const handleSelectNamespaceFile = async (fileKey: string) => {
  setSelectedNamespaceFile(fileKey);

  if (!fileKey) {
    setNamespacePreview({});
    return;
  }

  try {
    const parsed = await analyzeNamespaceFile(fileKey);
    setNamespacePreview({ parsedData: parsed });
  } catch (error) {
    setNamespacePreview({ error: 'プレビューの読み込みに失敗しました' });
  }
};
```

6-3. 重複チェックと確認ダイアログ表示

```
const handleConfirmImportWithDuplicateCheck = async () => {
  if (!selectedNamespaceFile) {
    message.error('テンプレートファイルを選択してください');
    return;
  }

  let parsed = namespacePreview.parsedData;
  if (!parsed) {
    try {
      parsed = await analyzeNamespaceFile(selectedNamespaceFile);
      setNamespacePreview({ parsedData: parsed });
    } catch (error) {
      message.error('名前空間の取り込みに失敗しました');
      return;
    }
  }

  const sourceData = namespaceImportMode === 'replace_all_properties'
    ? parsed.properties
    : parsed.contents;

  const duplicates = Object.keys(sourceData).filter(key =>
    namespaceImportMode === 'replace_all_properties' ? properties[key] : contents[key]
  );
  setDuplicateKeys(duplicates);
  setShowConfirmDialog(true);
};
```

6-4. 名前空間取り込み実行

```
const handleImportNamespace = () => {
  const parsed = namespacePreview.parsedData;
  if (!parsed) {
    message.error('取り込むデータがありません');
    return;
  }

  try {
    const templateType = namespaceImportMode === 'replace_all_properties' ? 'properties' :
'contents';

    if (templateType === 'properties') {
      replaceTemplateWithNamespace(
        parsed,
        'properties',
        (updater) => {
          if (typeof updater === 'function') {
            setProperties(prev => {
              const fakeTemplate = { properties: prev, contents };
              const next = updater(fakeTemplate as never);
              return (next as { properties: Record<string, PropertyData> }).properties;
            });
          }
        }
      );
      setForceRenderKey(Date.now());
      message.success(`${Object.keys(parsed.properties).length} 個のプロパティを取り込みました`);
    } else {
      replaceTemplateWithNamespace(
        parsed,
        'contents',
        (updater) => {
          if (typeof updater === 'function') {
            setContents(prev => {
              const fakeTemplate = { properties, contents: prev };
              const next = updater(fakeTemplate as never);
              return (next as { contents: Record<string, PropertyData> }).contents;
            });
          }
        }
      );
      setForceContentRenderKey(Date.now());
      message.success(`${Object.keys(parsed.contents).length} 個のContentsを取り込みました`);
    }

    const fileName = selectedNamespaceFile.split('/').pop() || selectedNamespaceFile;
    setImportedNamespaces(prev => [...prev, fileName]);
```

```
    setShowConfirmDialog(false);
    setShowNamespaceDialog(false);
    setSelectedNamespaceFile('');
    setNamespacePreview({});
    setDuplicateKeys([]);
    setHasChanges(true);
  } catch (error) {
    message.error(`取り込みに失敗しました: ${error instanceof Error ? error.message : '不明なエラー'}`);
  }
};
```

7. 保存処理

```
const handleSave = () => {
  // ✅ 基本チェック
  if (!templateId.trim()) {
    message.error('テンプレートIDを入力してください');
    return;
  }

  // ✅ ノードIDとの重複チェック
  const nodeIds = allNodes.map(n => n.id);
  if (nodeIds.includes(templateId) && templateId !== node.id) {
    message.error(
      `テンプレートID "${templateId}" は既にノードIDとして使用されています。別のIDを指定してください。`
    );
    return;
  }

  // ✅ 他のテンプレートIDとの重複チェック
  if (window.clineAllTemplates) {
    const allTemplateIds: string[] = [];
    (['material', 'condition', 'result'] as const).forEach(category => {
      const templates = window.clineAllTemplates[category];
      if (templates && Array.isArray(templates)) {
        templates.forEach((t: Template) => {
          const currentTemplateId = node.data.template?.id || node.data.templateRef?.id;
          const templateId = t.id;
          if (templateId && templateId !== currentTemplateId) {
            allTemplateIds.push(templateId);
          }
        });
      }
    });
```

```
    if (allTemplateIds.includes(templateId)) {
      message.error(
        `テンプレートID "${templateId}" は既に他のテンプレートで使用されています。別のIDを指定してくださ
い。`
      );
      return;
    }
  }

  // PropertyData形式からProperty[]とContent[]に変換
  const convertedProperties = convertPropertyDataToPropertiesArray(properties);
  const convertedContents = convertPropertyDataToContentsArray(contents);

  const templateRef: Template = {
    ...(node.data.templateRef || {}),
    id: templateId,
    uuid: uuid,
    name: name,
    category: nodeCategory,
    description: description,
    properties: convertedProperties,
    contents: convertedContents,
    placeRefs: placeRefs,
  };

  if (node.data.templateRef?.templateRef) {
    templateRef.templateRef = node.data.templateRef?.templateRef;
  }

  if ((node.data.placeType === 'material' || node.data.placeType === 'result') &&
      (insertion.uri || insertion.hash || insertion.format)) {
    templateRef.insertion = insertion;
  }

  const isNewTemplate = node.data.isNewTemplate || false;
  const templateCategory = node.data.templateCategory || node.data.placeType || 'material';
  const isCommonTemplate = node.data.isCommonTemplate || false;
  const methodId = node.data.methodId || null;

  onSave(node.id, templateRef, isNewTemplate, templateCategory, isCommonTemplate, methodId);
  onClose();
  setHasChanges(false);
};
```

保存時の処理フロー:

1. テンプレートID必須チェック
2. ノードIDとの重複チェック
3. 他テンプレートIDとの重複チェック（window.clineAllTemplates）

4. PropertyData形式 → Property[]配列に変換（`convertPropertyDataToPropertiesArray`）
5. PropertyData形式 → Content[]配列に変換（`convertPropertyDataToContentsArray`）
6. Templateオブジェクトを構築
7. Material/ResultのみInsertionを含める
8. 親コンポーネントのonSaveコールバックを呼び出し
9. ダイアログを閉じる

サブモーダル構成

1. プロパティ追加モーダル

トリガー: [新規プロパティ追加]ボタンクリック

UI要素:

- Input: プロパティキー入力
- Select: タグ名選択（property/content/parameter）
- [キャンセル]/[OK]ボタン

動作:

- プロパティキーを入力してOKクリック
- 重複チェック → 成功 → propertiesに追加
- モーダルを閉じる

2. Content追加モーダル（Resultテンプレートのみ）

トリガー: [新規Content追加]ボタンクリック

UI要素:

- Input: Contentキー入力
- Select: タグ名選択（property/content/parameter）
- [キャンセル]/[OK]ボタン

動作:

- Contentキーを入力してOKクリック
- 重複チェック → 成功 → contentsに追加
- モーダルを閉じる

3. PlaceRef追加ダイアログ

トリガー: [追加]ボタンクリック（Place Referencesセクション）

UI要素:

- Alert: 「既存のPlaceから選択してください」
- Place一覧（既に登録済みのものを除外）
  - Place ID
  - Place名
  - Place種別
  - [追加]ボタン（各Place）
- [閉じる]ボタン

動作:

- allNodesからnodeType='place'のものを抽出
- 既にplaceRefsに登録済みのものは表示しない
- Place選択 → placeRefsに追加
- メッセージ表示: 「Place Referenceを追加しました」
- ダイアログを閉じる

4. 名前空間取り込みモーダル

トリガー: [テンプレートから取り込み]ボタンクリック

UI要素:

- Alert: 取り込みの説明
  - 選択したテンプレートファイルのproperties定義を追加
  - 同じキーのプロパティは上書き
  - 複数のテンプレートから取り込み可能

- TreeSelect: テンプレートファイル選択
  - 部門 / フォルダ / ファイル の階層構造
  - 検索機能
- プレビュー表示（ファイル選択時）
  - プロパティ件数 / コンテンツ件数
  - キー一覧（xsi:type表示）
- [キャンセル]/[取り込み]ボタン

動作:

1. ダイアログ表示時にfetchTemplateFiles()でファイル一覧取得
2. ファイル選択 → analyzeNamespaceFile()でプレビュー取得
3. [取り込み]クリック → 重複チェック → 確認ダイアログ表示

5. 名前空間取り込み確認ダイアログ

トリガー: 名前空間取り込みモーダルで[取り込み]ボタンクリック

UI要素:

- Alert: 重複警告（重複がある場合）
  - 重複件数表示
  - 重複キー一覧（Tagで表示）
- または Alert: 「重複なし」（成功メッセージ）
- 取り込む要素の一覧
  - キー名
  - 重複の場合は「(上書き)」表示
- [キャンセル]/[取り込む]ボタン

動作:

1. 重複チェック結果を表示
2. [取り込む]クリック → handleImportNamespace()実行
3. replaceTemplateWithNamespace()で全置き換え + namespaceSource付与

4. 成功メッセージ表示: 「N個のプロパティを取り込みました」
5. ダイアログを閉じる

API連携

使用API

fetchTemplateFiles()

```
import { fetchTemplateFiles, type ListFormatsResponse } from '@/services/templateApi';
```

機能: テンプレートファイル一覧を取得

レスポンス:

```
interface ListFormatsResponse {
  departments: Array<{
    departmentCd: string;
    folders: Array<{
      folderName: string;
      files: Array<{
        fileName: string;
        key: string;
      }>;
    }>;
  }>;
}
```

使用箇所: 名前空間取り込みモーダル表示時

ユーティリティ関数

データ変換関数

convertPropertiesArrayToNestedFormat()

```
import {
  convertPropertiesArrayToNestedFormat,
  convertPropertiesArrayToNestedFormatRecursive,
  convertPropertyDataToPropertiesArray,
  convertPropertyDataToContentsArray,
  convertContentsArrayToNestedFormat,
} from '../utils/templateConverter';
```

機能: Property[]配列をNestedPropertyEditor用のRecord<string, PropertyData>形式に変換

使用箇所: 初期化時のProperties読み込み

convertContentsArrayToNestedFormat()

機能: Content[]配列をNestedPropertyEditor用のRecord<string, PropertyData>形式に変換

使用箇所: 初期化時のContents読み込み

convertPropertyDataToPropertiesArray()

機能: NestedPropertyEditor形式をProperty[]配列に変換（保存時）

使用箇所: 保存処理

convertPropertyDataToContentsArray()

機能: NestedPropertyEditor形式をContent[]配列に変換（保存時）

使用箇所: 保存処理

convertPropertiesArrayToNestedFormatRecursive()

機能: 継承元テンプレートのPropertiesを再帰的に変換（表示用）

使用箇所: 継承元プロパティ表示

関連コンポーネント

子コンポーネント

NestedPropertyEditor

- **ファイルパス**: `frontend/src/components/editorv2/components/NestedPropertyEditor.tsx`
- **用途**: プロパティの階層構造を表示・編集
- **Props:**

```
interface NestedPropertyEditorProps {
  propertyData: PropertyData;
  propertyKey: string;
  level: number;
  expandedNodes: Set<string>;
  onToggleExpanded: (nodeId: string) => void;
  onAddNestedChild: (parentKey: string, childTagName: string, specifiedKey?: string) =>
void;
  onAddSameLevelCopy: (sourceData: PropertyData, newKey: string) => void;
  onUpdateNestedAttribute: (propertyKey: string, attrName: string, attrValue: string) =>
void;
  onUpdateNestedElement: (propertyKey: string, elemName: string, elemValue: string) =>
void;
  onDeleteNestedAttribute: (propertyKey: string, attrName: string) => void;
  onDeleteNestedElement: (propertyKey: string, elemName: string) => void;
  onDeleteProperty: (propertyKey: string) => void;
  forceRenderKey: number;
}
```

カスタムフック

useNamespaceOperations

- **ファイルパス**: `frontend/src/components/editorv2/hooks/useNamespaceOperations.ts`
- **用途**: 名前空間の解析と取り込み処理
- **メソッド:**
  - `analyzeNamespaceFile(fileKey: string): Promise<NamespaceData>` - テンプレートファイルを解析

- `replaceTemplateWithNamespace(parsed, type, setEditingTemplate)` - 名前空間でテンプレートを置き換え

型定義

PropertyData

```
interface PropertyData {
  key: string;
  tagName: string;
  attributes: Record<string, string>;
  elements: Record<string, string>;
  children?: PropertyData[];
}
```

NamespaceData

```
interface NamespaceData {
  properties: Record<string, PropertyData>;
  contents: Record<string, PropertyData>;
}
```

Template（MAIMLテンプレート）

```
import { Template, Insertion, PlaceRef } from '@/types/maiml-unified';
```

親コンポーネント

PetriNetEditorInner

- **ファイルパス**: `frontend/src/components/editorv2/PetriNetEditorInner.tsx`
- **用途**: MAIML Petri Netエディタのメイン画面

- **連携**: Placeノードダブルクリックで TemplateEditorDialog を開く

データ型定義

PropertyData（NestedPropertyEditor用）

```
interface PropertyData {
  key: string;                          // プロパティキー
  tagName: string;                      // XMLタグ名（property/content/parameter）
  attributes: Record<string, string>;   // 属性（xsi:type等）
  elements: Record<string, string>;     // 要素（name, value等）
  children?: PropertyData[];            // 子要素（ネスト構造）
}
```

Template（MAIML）

```
interface Template {
  id: string;
  uuid?: string;
  name?: string;
  description?: string;
  category?: string;
  properties?: Property[] | Record<string, PropertyData>;
  contents?: Content[] | Record<string, PropertyData>;
  insertion?: Insertion;
  placeRefs?: PlaceRef[];
  templateRef?: TemplateRef;
}
```

Insertion（Material/Result用）

```
interface Insertion {
  uri: string;
  hash?: {
    method: string;
    value: string;
```

```
  };
  format: string;
}
```

PlaceRef

```
interface PlaceRef {
  id?: string;
  ref: string;  // Place ID
}
```

エラーハンドリング

バリデーションエラー表示

- Props `templateErrors` から該当テンプレートのエラー一覧を取得
- Alert（warning）でエラー件数とメッセージを表示
- エラータイプと詳細メッセージをリスト表示

入力チェック

- テンプレートID: 必須、ノードIDとの重複禁止、他テンプレートIDとの重複禁止
- UUID: 形式チェック（オプション）
- プロパティキー: 必須、重複チェック
- Contentキー: 必須、重複チェック

API通信エラー

- fetchTemplateFiles失敗時: 「テンプレートファイル一覧の取得に失敗しました」
- analyzeNamespaceFile失敗時: 「プレビューの読み込みに失敗しました」
- 名前空間取り込み失敗時: 「取り込みに失敗しました: {エラーメッセージ}」

パフォーマンス最適化

強制再レンダリング

- `forceRenderKey` / `forceContentRenderKey` を使用
- Date.now()で一意のキーを生成
- NestedPropertyEditorの深いネスト更新時に必要

不変性の保持

- Properties/Contents更新時は `JSON.parse(JSON.stringify())` で深いコピー
- 関数型setState（ `prev => ...` ）を使用

展開状態の管理

- `expandedNodes` / `expandedContentNodes` をSetで管理
- 追加/削除時のパフォーマンスが良い

変更履歴

日付	バージョン	変更内容	担当者
2026-03-11	1.0.0	実装ベースでUI仕様書を新規作成	Cline
2026-01-30	-	Phase 2-1: NestedPropertyEditor実装完了（実装）	-
2025-XX-XX	-	EditorV2用Template編集ダイアログ初版作成（実装）	-

実装ファイル情報

- **ファイルパス**: `frontend/src/components/editorv2/components/TemplateEditorDialog.tsx`
- **コンポーネント名**: `TemplateEditorDialog`

- **実装バージョン**: 2.1.0
- **実装日**: 2026-01-30